

Imitation Learning with Latent Market Parameters in Continuous-Time Portfolio Control

SSC Annual Conference 2026

Mark Frankli¹

¹Wilfrid Laurier University, Department of Mathematics

June 2026

Motivation & Problem Setting

- Portfolio optimization is a fundamental task in quantitative finance.
- Traditional methods have strong assumptions and are less flexible.
- Machine Learning (and, in particular, Reinforcement Learning) has been applied to finance with great success.
- In portfolio optimization our goal is to distribute (and then potentially reallocate at each time step) our wealth into n assets to maximize a utility function.
- We will consider the $n = 2$ case: a risky asset with mean μ and volatility σ and a risk-free asset with constant return r .

Research Questions & Contributions

The questions we want to answer are:

- ❶ Can RL-based algorithms recover the expert policy?
- ❷ Under what circumstances can imitation learning be a better choice?
- ❸ How can imitation learning enhance well-established RL algorithms such as PPO?

Our Contributions

- **Contribution 1:** Rigorous comparison of Imitation Learning-based algorithms to traditional optimal control solutions.
- **Contribution 2:** An Imitation Learning-based algorithm that uses RNNs to outperform the baseline method using the Merton model.
- **Contribution 3:** Hot-starting PPO with the Neural Network learned by our algorithm stabilizes and improves training (transfer learning).

RL Background

The two Imitation Learning algorithms we will use are Behavior Cloning (BC) and Dataset Aggregation (DAGGER).

BC

- 1: Initialize $\mathcal{D}$ with a set of expert trajectories
- 2: $\max_{\pi_{\theta}} \mathbb{E}_{(s,a) \sim \mathcal{D}} [\log \pi_{\theta}(a|s)]$ using gradient descent

DAGGER

- 1: Initialize $\mathcal{D} \leftarrow \emptyset$ and $\hat{\pi}_1 \leftarrow$ random policy
- 2: **for** $i = 1..N$ **do**
- 3: $\pi_i(a|s) \leftarrow \beta_i \pi^*(a|s) + (1 - \beta_i) \hat{\pi}_i(a|s)$
- 4: $D_i \leftarrow \{(s, \pi^*(s))\}$ of visited states by π_i and actions given by the teacher
- 5: $D \leftarrow D \cup D_i$
- 6: Train classifier $\hat{\pi}_{i+1}$ on D
- 7: **end for**

Merton Model

There are two financial models that we will use as expert policies. The first one is the stochastic control model by Merton (1969).

Assumption (Wealth movements)

$$\frac{dX_t}{X_t} = [(\mu - r)\pi_t + r]dt + \pi_t\sigma dW_t$$

Objective (Objective function of the Merton model)

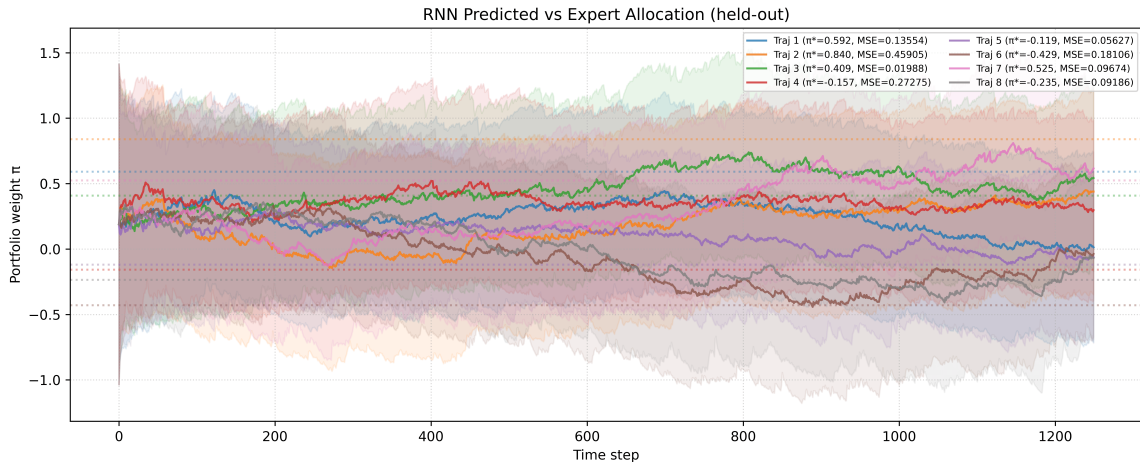
$$\max \mathbb{E} \left[\int_0^T U(C(t))dt + B(X_T) \right]$$

where C is the consumption, U is the utility function and B is assumed to be concave. Under CRRA utility ($U(x) = x^{1-\gamma}/(1-\gamma)$), the optimal solution is $\pi_t^* = (\mu - r)/(\gamma\sigma^2)$. We will consider the terminal wealth problem (consume everything at T).

Trajectory-dependent Merton Model

- Learning the optimal policy when μ and σ are fixed for all trajectories is easy even with a linear policy.
- So, let's switch to a more complex model: the mean and volatility of the risky asset are random per-trajectory.
- $\mu_k \sim \mathcal{N}(\mu, \sigma_d^2)$ and $\sigma_k \sim \text{LogNormal}(\log \sigma, \sigma_d^2)$ are generated randomly at the beginning of the k_{th} trajectory and then kept fixed for the entire trajectory.
- Since it is a much harder problem, we first use Neural Networks as our policy.
- $s_t = (t/N, R_t, \log X_t)$ does not work with feed-forward NNs $\rightarrow$ include sufficient sample statistics $\hat{\mu}_t$ and $\hat{\sigma}_t$.
- This works better but still too volatile.

Performance of RNN-based Policy (Merton Model)



Jump-diffusion Models

Second model used is more complex, allows occasional "shocks" in stock prices.

Assumption (Price movements)

$$\frac{dS_t}{S_t} = \mu_{eff}dt + \sigma dW_t + (J - 1)dN_t$$

There is no closed form solution to the stochastic optimization problem, we can only approximate π^* .

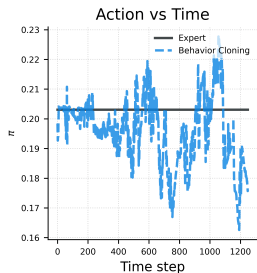
Solution (Second-order approximated solution)

$$\pi^* \approx \frac{\mu - r}{\gamma(\sigma^2 + \lambda \mathbb{E}[(J - 1)^2])}$$

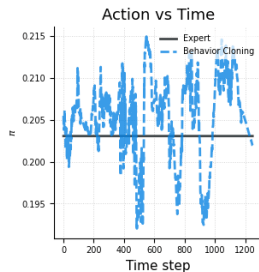
We will do the same analysis as with the Merton model.

BC vs. DAgger using Jump-diffusion Models

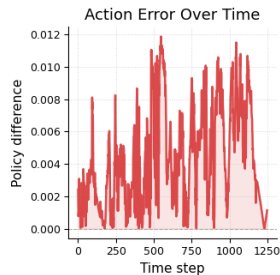
- DAgger was not necessary when using the Merton model.
- Jump-diffusion models are more complex, recovering even a constant policy is non-trivial.
- DAgger improves both accuracy and error accumulation.



BC

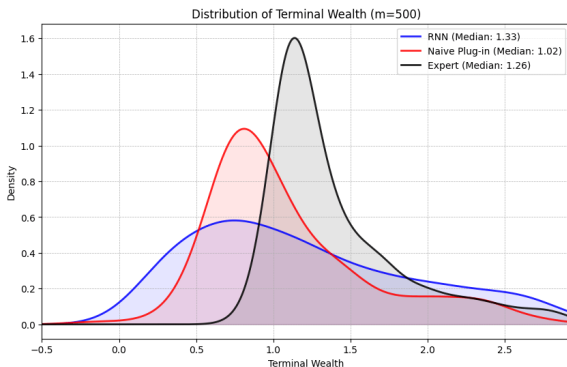


DAgger



RNN vs. Naive Policy

- We do not know the per-trajectory parameters but want to use the Merton formula.
- Need to estimate the parameters: naive policy uses sample mean and variance.
- RNN policy achieves significantly higher utility compared to the naive policy while sacrificing some of its left tail.



Policy	Mean	Median	Std
RNN	2.088	1.334	2.278
Naive	2.880	1.023	15.426
Expert	1.997	1.261	3.625

Policy	P5	P95	CRRA Utility
RNN	0.281	5.896	-36.468
Naive	0.508	5.999	-2.5×10^{45}
Expert	1.016	4.102	-0.104

Pre-training PPO

- PPO is a well-known RL algorithm that has been used in financial applications.
- Pre-trained PPO is close to optimal and has higher median terminal wealth while sacrificing its left tail.
- Randomly initialized PPO has extremely low CRRA utility.

